NLP Lab: Trigrams and Word Embeddings

A Guided Introduction to Statistical and Neural NLP

Natural Language Processing / Computational Linguistics

Course Assignment — Jurafsky & Martin, Ch. 2, 3, & 6

Learning Objectives

By completing this lab, you will:

1. Preprocess a small text corpus.
2. Build basic n-gram counts.
3. Calculate trigram probabilities.
4. Understand perplexity.
5. Train and inspect a Word2Vec Skip-gram model.
6. Compare n-gram models with word embeddings.

Dataset

NLTK `webtext` corpus — specifically `wine.txt`

Libraries

`nltk`, `re`, `collections.Counter`, `math`, `gensim`

Setup

In [16]:
```
import nltk
import re
from collections import Counter
import math
from gensim.models import Word2Vec

nltk.download('webtext', quiet=True)
from nltk.corpus import webtext
```

Task 1: Preprocessing

Write a function that converts the raw corpus into a list of tokenized sentences.

Your function should:

1. Split the text into lines.
2. Convert text to lowercase.
3. Remove punctuation.
4. Return tokens for each non-empty line.

Hints: `str.split()`, `str.strip()`, `str.lower()`, `re.findall()` with pattern `r'\w+'`

Required output: Print the number of processed sentences.

In [17]:
```
def preprocess(raw_text):
    """Convert raw corpus into a list of tokenized sentences."""
    sentences = []
    for line in raw_text.split('\n'):
        line = line.strip().lower()
        tokens = re.findall(r'\w+', line)
        if tokens:
            sentences.append(tokens)
    return sentences
```

In [18]:
```
raw = webtext.raw('wine.txt')
sentences = preprocess(raw)

print(f"Number of processed sentences: {len(sentences)}")
print(f"\nFirst 3 sentences (preview):")
for s in sentences[:3]:
    print(f"  {s}")
```

```
Number of processed sentences: 1230

First 3 sentences (preview):
  ['lovely', 'delicate', 'fragrant', 'rhone', 'wine', 'polished', 'leather',
'and', 'strawberries', 'perhaps', 'a', 'bit', 'dilute', 'but', 'good', 'fo
r', 'drinking', 'now']
  ['liquorice', 'cherry', 'fruit', 'simple', 'and', 'coarse', 'at', 'the',
'finish']
  ['thin', 'and', 'completely', 'uninspiring']
```

Task 2: Trigram Counts

Create a simple trigram language model using the training portion of the corpus.

First, split the corpus into:

- 80% training data
- 20% test data

For each training sentence, add:

- `<s> <s>` at the beginning and `</s>` at the end.

Build:

- Unigram counts
- Bigram counts
- Trigram counts

Use a minimum frequency of 3 to replace rare words with `<UNK>`.

Hints: `collections.Counter`, `Counter.update()`, tuple keys such as `(w1, w2)`, `range()`

In [19]:
```
# ── Train / Test split (80/20) ──
split_idx = int(len(sentences) * 0.8)
train_sentences_raw = sentences[:split_idx]
test_sentences_raw  = sentences[split_idx:]

print(f"Train sentences: {len(train_sentences_raw)}")
print(f"Test sentences:  {len(test_sentences_raw)}")
```

```
Train sentences: 984
Test sentences:  246
```

In [20]:
```
# ── Build vocabulary with min frequency = 3; replace rare words with <UNK>
MIN_FREQ = 3

word_freq = Counter()
for sent in train_sentences_raw:
    word_freq.update(sent)

vocab = {w for w, c in word_freq.items() if c >= MIN_FREQ}
vocab.update(['<s>', '</s>', '<UNK>'])
vocab_size = len(vocab)

print(f"Words before <UNK> replacement: {len(word_freq)}")
print(f"Vocabulary size (freq >= {MIN_FREQ} + special tokens): {vocab_size}"
```

```
Words before <UNK> replacement: 2550
Vocabulary size (freq >= 3 + special tokens): 897
```

In [21]:
```
def replace_unk(sentence, vocabulary):
    """Replace words not in vocabulary with <UNK>."""
    return [w if w in vocabulary else '<UNK>' for w in sentence]

def pad_sentence(sentence):
    """Add <s> <s> at the beginning and </s> at the end."""
    return ['<s>', '<s>'] + sentence + ['</s>']

# Apply <UNK> replacement and padding to train sentences
train_sentences = [pad_sentence(replace_unk(s, vocab)) for s in train_senter
# Apply <UNK> replacement and padding to test sentences (using train vocab)
```

```
test_sentences  = [pad_sentence(replace_unk(s, vocab)) for s in test_sentenc

print(f"Example padded train sentence:")
print(f"  {train_sentences[0]}")
```

```
Example padded train sentence:
  ['<s>', '<s>', 'lovely', 'delicate', 'fragrant', 'rhone', 'wine', '<UNK>',
'leather', 'and', 'strawberries', 'perhaps', 'a', 'bit', 'dilute', 'but', 'g
ood', 'for', 'drinking', 'now', '</s>']
```

In [22]:
```
# ── Build unigram, bigram, and trigram counts from training data ──
unigram_counts = Counter()
bigram_counts  = Counter()
trigram_counts = Counter()

for sent in train_sentences:
    for i in range(len(sent)):
        unigram_counts[(sent[i],)] += 1
        if i >= 1:
            bigram_counts[(sent[i-1], sent[i])] += 1
        if i >= 2:
            trigram_counts[(sent[i-2], sent[i-1], sent[i])] += 1

# Total token count for unigram probability denominator
total_unigrams = sum(unigram_counts.values())

print(f"Unique unigrams: {len(unigram_counts)}")
print(f"Unique bigrams:  {len(bigram_counts)}")
print(f"Unique trigrams: {len(trigram_counts)}")
print(f"Total tokens:    {total_unigrams}")
```

```
Unique unigrams: 897
Unique bigrams:  9536
Unique trigrams: 16528
Total tokens:    22782
```

In [23]:
```
print("Top 10 unigrams:")
for gram, count in unigram_counts.most_common(10):
    print(f"  {gram}: {count}")

print("\nTop 10 bigrams:")
for gram, count in bigram_counts.most_common(10):
    print(f"  {gram}: {count}")

print("\nTop 10 trigrams:")
for gram, count in trigram_counts.most_common(10):
    print(f"  {gram}: {count}")
```

```
Top 10 unigrams:
  ('<UNK>',): 2025
  ('<s>',): 1968
  ('</s>',): 984
  ('a',): 858
  ('and',): 654
  ('the',): 603
  ('of',): 414
  ('but',): 392
  ('i',): 319
  ('good',): 299

Top 10 bigrams:
  ('<s>', '<s>'): 984
  ('<UNK>', '<UNK>'): 274
  ('a', 'bit'): 176
  ('top', '</s>'): 151
  ('the', '<UNK>'): 148
  ('bare', '</s>'): 109
  ('<s>', '<UNK>'): 104
  ('a', '<UNK>'): 101
  ('<UNK>', 'and'): 97
  ('a', 'touch'): 96

Top 10 trigrams:
  ('<s>', '<s>', '<UNK>'): 104
  ('<s>', '<s>', 'a'): 73
  ('<s>', '<s>', 'very'): 49
  ('not', 'rated', '</s>'): 43
  ('<s>', '<s>', 'quite'): 42
  ('<UNK>', '<UNK>', '<UNK>'): 40
  ('a', 'touch', 'of'): 37
  ('a', 'bit', '<UNK>'): 34
  ('<s>', 'a', 'bit'): 29
  ('a', 'good', '</s>'): 26
```

Task 3: Trigram Probability

Implement a function that calculates the **MLE trigram probability**:

$$P(w_3 \mid w_1, w_2) = \frac{C(w_1, w_2, w_3)}{C(w_1, w_2)}$$

If the history has never occurred, return probability zero.

Then implement **linear interpolation**:

$$P_{\text{interp}} = 0.6 \cdot P(w_3 \mid w_1, w_2) + 0.3 \cdot P(w_3 \mid w_2) + 0.1 \cdot P(w_3)$$

Required output: Calculate $P(\texttt{tannins} \mid \texttt{firm}, \texttt{ripe})$ using the interpolated model.

In [24]:
```python
def trigram_prob(w1, w2, w3):
    """MLE trigram probability: P(w3 | w1, w2)."""
    denom = bigram_counts.get((w1, w2), 0)
    if denom == 0:
        return 0.0
    return trigram_counts.get((w1, w2, w3), 0) / denom

def bigram_prob(w2, w3):
    """MLE bigram probability: P(w3 | w2)."""
    denom = unigram_counts.get((w2,), 0)
    if denom == 0:
        return 0.0
    return bigram_counts.get((w2, w3), 0) / denom

def unigram_prob(w3):
    """MLE unigram probability: P(w3)."""
    return unigram_counts.get((w3,), 0) / total_unigrams

def interpolated_prob(w1, w2, w3, lambda1=0.6, lambda2=0.3, lambda3=0.1):
    """Linear interpolation of trigram, bigram, and unigram probabilities."""
    p3 = trigram_prob(w1, w2, w3)
    p2 = bigram_prob(w2, w3)
    p1 = unigram_prob(w3)
    return lambda1 * p3 + lambda2 * p2 + lambda3 * p1
```

In [25]:
```python
# ── Required output: P(tannins | firm, ripe) ──
w1, w2, w3 = 'firm', 'ripe', 'tannins'

p_tri = trigram_prob(w1, w2, w3)
p_bi  = bigram_prob(w2, w3)
p_uni = unigram_prob(w3)
p_int = interpolated_prob(w1, w2, w3)

print(f"P(tannins | firm, ripe):")
print(f"  Trigram  MLE:       P = {p_tri:.6f}")
print(f"  Bigram   MLE:       P = {p_bi:.6f}")
print(f"  Unigram  MLE:       P = {p_uni:.6f}")
print(f"  Interpolated:       P = {p_int:.6f}")
print(f"    = 0.6*{p_tri:.6f} + 0.3*{p_bi:.6f} + 0.1*{p_uni:.6f}")
```

```
P(tannins | firm, ripe):
  Trigram  MLE:       P = 0.000000
  Bigram   MLE:       P = 0.085714
  Unigram  MLE:       P = 0.001492
  Interpolated:       P = 0.025864
    = 0.6*0.000000 + 0.3*0.085714 + 0.1*0.001492
```

Task 4: Perplexity

Calculate the perplexity of your trigram model on the test set using the **interpolated** model.

$$\log_2 PP = -\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i \mid w_{i-2}, w_{i-1})$$

$$PP = 2^{\log_2 PP}$$

If a probability is zero, use a small floor such as 10^{-10} before taking the logarithm.

Required output: Report the test-set perplexity.

In [26]:
```
def compute_perplexity(padded_test_sentences):
    """Compute perplexity of the interpolated trigram model on test sentence
    Sentences are already padded with <s> <s> ... </s>.
    """
    log_prob_sum = 0.0
    N = 0

    for sent in padded_test_sentences:
        # Start at index 2 — predict each word given its two predecessors
        for i in range(2, len(sent)):
            prob = interpolated_prob(sent[i-2], sent[i-1], sent[i])
            if prob == 0:
                prob = 1e-10   # floor to avoid log2(0)
            log_prob_sum += math.log2(prob)
            N += 1

    log2_pp = -log_prob_sum / N
    perplexity = 2 ** log2_pp
    return perplexity, N
```

In [27]:
```
pp, N = compute_perplexity(test_sentences)

print(f"Test set tokens (N): {N}")
print(f"Test-set perplexity (interpolated model): {pp:.2f}")
```

```
Test set tokens (N): 4860
Test-set perplexity (interpolated model): 144.64
```

Task 5: Word2Vec

Train a Word2Vec **Skip-gram** model using the training sentences.

- Remove the sentence-boundary markers (`<s>` , `</s>`) before training.
- Configuration: `vector_size=100` , `window=5` , `sg=1` , `min_count=2`

Required output: Report the five nearest neighbors of **tannins** and their similarity scores.

In [28]:
```
# Remove sentence-boundary markers for Word2Vec training
w2v_train = [[w for w in sent if w not in ('<s>', '</s>')] for sent in trair
```

```
model = Word2Vec(
    sentences=w2v_train,
    vector_size=100,
    window=5,
    sg=1,           # Skip-gram
    min_count=2,
    epochs=50,
    seed=42,
    workers=1
)

print(f"Word2Vec vocabulary size: {len(model.wv)}")
print(f"Embedding dimension:      {model.wv.vector_size}")
```

```
Word2Vec vocabulary size: 895
Embedding dimension:      100
```

In [29]:
```
# ── Required output: 5 nearest neighbors of "tannins" ──
target_word = 'tannins'

if target_word in model.wv:
    neighbors = model.wv.most_similar(target_word, topn=5)
    print(f"5 most similar words to '{target_word}':")
    for word, score in neighbors:
        print(f"  {word:20s}  similarity = {score:.4f}")
else:
    print(f"'{target_word}' not in Word2Vec vocabulary")
```

```
5 most similar words to 'tannins':
  succulent            similarity = 0.5758
  classy               similarity = 0.5138
  gives                similarity = 0.5050
  evident              similarity = 0.4977
  softer               similarity = 0.4888
```

In [30]:
```
# ── Additional exploration: more wine-related words ──
extra_words = ['wine', 'red', 'white', 'dry', 'sweet']

print("Most similar words (top 5 each):")
for word in extra_words:
    if word in model.wv:
        similar = model.wv.most_similar(word, topn=5)
        sim_str = ', '.join([f"{w} ({s:.3f})" for w, s in similar])
        print(f"  '{word}': {sim_str}")
    else:
        print(f"  '{word}': not in vocabulary")
```

```
Most similar words (top 5 each):
  'wine': getting (0.485), pichon (0.465), taste (0.446), sit (0.414), usual
ly (0.414)
  'red': fruits (0.763), bright (0.674), mainly (0.607), dinner (0.582), sop
histicated (0.574)
  'white': bread (0.699), summer (0.665), pepper (0.659), flowers (0.628), p
lus (0.586)
  'dry': flavoury (0.555), taught (0.534), cherries (0.513), mouthwatering
(0.507), bone (0.506)
  'sweet': raisiny (0.545), bags (0.543), ripe (0.542), cigar (0.538), barle
y (0.525)
```

Task 6: Short Analysis

1. Why can a trigram model assign probability zero to a reasonable phrase that was not observed during training?

A trigram model estimates $P(w_3 \mid w_1, w_2)$ directly from counts: $C(w_1, w_2, w_3)/C(w_1, w_2)$. If the specific three-word sequence was never seen in the training data, the count $C(w_1, w_2, w_3) = 0$, making the probability exactly zero — regardless of whether the phrase is linguistically reasonable. The model has no mechanism to generalize beyond what it has explicitly observed. This is the **sparsity problem**: the space of possible trigrams grows as V^3, so even a large training corpus will leave most valid trigrams unseen.

2. Why does taking $\log_2(0)$ cause a problem when calculating perplexity?

$\log_2(0)$ is mathematically undefined (it tends to $-\infty$). In the perplexity formula, we sum $\log_2 P(w_i \mid \text{context})$ over all test tokens. A single zero-probability token sends the entire sum to $-\infty$, making the perplexity $2^{+\infty} = \infty$. This means one unseen trigram can make the model appear infinitely bad, even if it predicts every other word well. This is why smoothing techniques (Laplace, interpolation, etc.) or a probability floor are essential — they ensure no event receives exactly zero probability.

3. How can Word2Vec identify words related to *tannins* even when a particular word sequence never appeared in the training corpus?

Word2Vec learns dense vector representations by training on local context windows, not on specific n-gram sequences. Words that frequently appear in **similar contexts** (e.g., surrounded by words like "firm", "dry", "astringent", "oak") are pushed close together in the embedding space — even if they never co-occur directly. So *tannins* and a related word like *acidity* might both appear near descriptors of wine structure, giving them

similar vectors. This **distributional similarity** ("you shall know a word by the company it keeps" — Firth, 1957) allows Word2Vec to capture semantic relationships that count-based n-gram models completely miss.